

Phần 1: Khái niệm cơ bản

1. Backend là gì?

Nếu coi một trang web hay ứng dụng như một nhà hàng:

- **Front-end** là không gian quán, bàn ghế, thực đơn và người phục vụ – những gì khách hàng nhìn thấy và tương tác trực tiếp.
- **Back-end** chính là **khu vực nhà bếp**. Khách hàng không nhìn thấy bếp, nhưng đây là nơi xử lý nguyên liệu, nấu nướng và tạo ra món ăn.

Trong công nghệ, **Back-end** là phần chạy ngầm trên máy chủ (Server). Nó chịu trách nhiệm xử lý logic, tính toán, tương tác với cơ sở dữ liệu (Database) và đảm bảo ứng dụng hoạt động mượt mà, bảo mật. Khi bạn dùng Python làm Back-end, bạn sẽ viết code để xử lý các dữ liệu này.

2. Client – Server là gì?

Đây là mô hình hoạt động cơ bản của Internet, hiểu đơn giản là mối quan hệ giữa **Người mua (Client)** và **Người bán (Server)**.

- **Client (Khách hàng)**: Là thiết bị hoặc ứng dụng phía người dùng gửi yêu cầu. Ví dụ: Trình duyệt web (Chrome, Safari), ứng dụng Shopee trên điện thoại của bạn.
- **Server (Máy chủ)**: Là một máy tính mạnh mẽ, hoạt động 24/7, lưu trữ dữ liệu của trang web. Nó "nghe" các yêu cầu từ Client, xử lý rồi trả kết quả về cho Client.

Ví dụ: Bạn gõ google.com (Client gửi yêu cầu) -> Máy chủ Google nhận được (Server) -> Server trả về giao diện trang tìm kiếm cho bạn.

3. API là gì?

API (Application Programming Interface) đóng vai trò như **người bồi bàn** trong nhà hàng.

- Bạn (Client) không thể chạy thẳng vào bếp (Server) để lấy món ăn. Bạn cần nói với bồi bàn: *"Cho tôi một đĩa cơm chiên"*.
- Bồi bàn (API) mang yêu cầu đó vào bếp. Bếp làm xong, bồi bàn lại bung đĩa cơm ra cho bạn.

API là cầu nối cho phép hai ứng dụng hoặc các thành phần của hệ thống giao tiếp, trao đổi dữ liệu với nhau. Khi bạn viết Back-end bằng Python, việc chính của bạn thường là tạo ra các API để Front-end gọi vào lấy dữ liệu.

4. JSON là gì?

JSON (JavaScript Object Notation) là một **định dạng văn bản dùng để lưu trữ và truyền tải dữ liệu**. Hãy tưởng tượng nó như một "ngôn ngữ chung" cực kỳ đơn giản mà cả Front-end, Back-end hay bất kỳ ngôn ngữ lập trình nào (Python, JavaScript, Java...) đều đọc hiểu được.

Dữ liệu JSON được viết dưới dạng các cặp key: value (khóa: giá trị), nhìn rất giống kiểu dữ liệu Dictionary trong Python.

Phần 2: Kiến thức Web cơ bản

1. HTTP là gì?

HTTP (Hypertext Transfer Protocol) là **bộ quy tắc giao tiếp** (giao thức) giữa Client và Server. Nếu API là người bồi bàn, thì HTTP chính là *ngôn ngữ/nghi thức* mà khách hàng và bồi bàn dùng để nói chuyện với nhau, tránh việc người nói tiếng Anh, người nói tiếng Việt dẫn đến hiểu lầm.

Khi bạn lướt web, mọi hành động gửi nhận dữ liệu đều tuân theo giao thức HTTP này.

2. GET / POST / PUT / DELETE dùng để làm gì?

Trong giao thức HTTP, đây được gọi là các **HTTP Methods (Phương thức HTTP)**. Chúng đại diện cho các hành động mà Client muốn thực hiện đối với dữ liệu trên Server (thường được gọi là chuẩn **CRUD** - Create, Read, Update, Delete).

Phương thức	Hành động tương ứng	Ví dụ thực tế
GET	Đọc/Lấy dữ liệu về (Read)	Bạn vào Facebook để xem bảng tin, đọc bài viết.
POST	Tạo mới dữ liệu (Create)	Bạn nhấn "Đăng bài" viết một status mới, hoặc đăng ký tài khoản.
PUT	Cập nhật/Sửa dữ liệu cũ (Update)	Bạn sửa lại nội dung bài viết đã đăng hoặc đổi avatar.
DELETE	Xóa dữ liệu (Delete)	Bạn bấm nút "Xóa bài viết".

Phần 3: Tìm hiểu FastAPI

1. FastAPI là gì?

FastAPI là một **Web Framework** hiện đại, có hiệu năng rất cao dùng để xây dựng các API bằng Python. Đúng như cái tên của nó ("Fast"), framework này được thiết kế để giúp lập trình viên viết code cực nhanh, dễ học và khi chạy thực tế thì tốc độ xử lý tối ưu gần như tương đương với các ngôn ngữ có tiếng là nhanh như Go hay Node.js.

2. FastAPI dùng để làm gì?

- **Xây dựng API cho ứng dụng:** Làm phần Back-end cốt lõi để cung cấp dữ liệu cho các ứng dụng web (React, Vue, Angular) hoặc ứng dụng di động (Flutter, Android, iOS).
- **Tự động tạo tài liệu (Documentation):** Một điểm "ăn tiền" của FastAPI là khi bạn viết code xong, nó tự động tạo ra một trang web tài liệu (Swagger UI) cực đẹp để các bạn làm Front-end nhìn vào là biết cách gọi API của bạn mà bạn không cần viết tay một dòng giải thích nào.
- **Xử lý các bài toán AI/Data Science:** Vì viết bằng Python, FastAPI cực kỳ phổ biến để làm Back-end cho các hệ thống Machine Learning, AI hoặc xử lý dữ liệu lớn.

3. FastAPI khác gì so với web framework truyền thống (như Django, Flask)?

Để bạn dễ hình dung, chúng ta cùng so sánh FastAPI với hai "đàn anh" nổi tiếng trong Python là Django và Flask:

- **So với Django (Framework "Mì ăn liền" - Toàn diện):** Django rất đồ sộ, tích hợp sẵn mọi thứ từ quản lý giao diện, cơ sở dữ liệu, bảo mật (gọi là *Batteries-included*). Django phù hợp cho các trang web truyền thống. Trong khi đó, **FastAPI tinh gọn hơn**, nó tập trung tối đa vào việc làm **API** và không can thiệp vào việc bạn chọn cơ sở dữ liệu hay giao diện nào.
- **So với Flask (Framework siêu nhỏ - Microframework):** Flask cũng tinh gọn như FastAPI, nhưng Flask đã khá cũ. **FastAPI vượt trội hơn Flask** nhờ hai yếu tố cốt lõi:
 - **Hỗ trợ Asynchronous (Bất đồng bộ - async/await):** Giúp Server xử lý hàng ngàn yêu cầu cùng một lúc mà không bị nghẽn (Flask truyền thống không mạnh khoản này).
 - **Type Hinting (Định nghĩa kiểu dữ liệu):** FastAPI ép hoặc khuyến khích bạn khai báo rõ biến nào là chuỗi (string), biến nào là số (int). Điều này giúp code ít lỗi hơn và tự động kiểm tra dữ liệu đầu vào (Validation) cực kỳ nghiêm ngặt, việc mà ở Flask bạn phải tự viết code kiểm tra rất mất thời gian.